

TÀI LIỆU THỰC HÀNH

LẬP TRÌNH SOCKET TCP VÀ UDP TRONG JAVA

1. Mục tiêu

Bài thực hành giúp sinh viên xây dựng được chương trình mạng theo mô hình client server, thiết kế giao thức thông điệp đơn giản và quan sát sự khác biệt giữa TCP với UDP. Các ví dụ dùng cổng 5000 trở lên, mã hóa UTF-8, try-with-resources và thông báo lỗi có ngữ cảnh.

Sau khi hoàn thành bài thực hành, sinh viên có thể:

- Tra cứu địa chỉ bằng InetAddress và phân tích URI theo từng thành phần.
- Mô tả vai trò của địa chỉ IP, cổng, socket và giao thức ứng dụng.
- Xây dựng TCP server, TCP client và trao đổi nhiều yêu cầu trên một kết nối.
- Phục vụ đồng thời nhiều TCP client bằng ExecutorService.
- Gửi và nhận datagram UDP, xử lý đúng chiều dài dữ liệu và timeout.
- Thiết kế ca kiểm thử cho dữ liệu đúng, dữ liệu sai, mất kết nối và nhiều client.
- Giải thích lựa chọn TCP hoặc UDP cho một yêu cầu cụ thể.

2. Điều chỉnh đề xuất từ nội dung ban đầu

Nội dung ban đầu	Điều chỉnh trong tài liệu mới	Lý do
Cổng 7 cho chương trình echo	Dùng 5000 cho TCP và 5001 cho UDP	Cổng 7 là cổng dịch vụ hệ thống đã đăng ký; cổng cao phù hợp hơn cho thực hành.
Đổi chuỗi và byte theo charset mặc định	Chỉ rõ StandardCharsets.UTF_8	Bảo toàn tiếng Việt và tạo kết quả nhất quán giữa các máy.
Tự đóng socket trong finally	Dùng try-with-resources	Giảm rò rỉ tài nguyên và đơn giản hóa xử lý lỗi.
Tạo một Thread cho mỗi client	Dùng thread pool có giới hạn	Kiểm soát tài nguyên khi nhiều client kết nối.
Không có timeout UDP	Thiết lập setSoTimeout	Client không chờ vô hạn khi gói tin bị mất hoặc server chưa chạy.
Phân lớp địa chỉ IP A B C	Chuyển thành IPv4 hoặc IPv6, loopback, private hoặc public	Phân lớp A B C chỉ còn ý nghĩa lịch sử; mạng hiện đại dùng CIDR.
URL constructor và URLConnection cơ bản	Ưu tiên URI để phân tích; HttpClient là phần mở rộng	Tách rõ phân tích cú pháp với truy cập tài nguyên mạng.
Danh sách bài tập chưa có tiêu chí	Bổ sung đầu vào, đầu ra, kiểm thử và thang điểm	Giúp sinh viên tự kiểm tra và giảng viên đánh giá nhất quán.

3. Kiến thức nền tối thiểu

3.1. Thành phần của một ứng dụng socket

Thành phần	Ý nghĩa	Ví dụ trong bài
Địa chỉ IP hoặc hostname	Xác định máy đích trên mạng	127.0.0.1 hoặc localhost
Port	Xác định tiến trình dịch vụ trên máy	5000 hoặc 5001

Thành phần	Ý nghĩa	Ví dụ trong bài
Socket	Điểm cuối dùng để gửi hoặc nhận dữ liệu	Socket, ServerSocket, DatagramSocket
Giao thức ứng dụng	Quy định cú pháp và ý nghĩa thông điệp	PING, UPPER văn bản, TIME, QUIT
Charset	Quy tắc chuyển đổi giữa ký tự và byte	UTF-8

3.2. So sánh TCP và UDP

Tiêu chí	TCP	UDP
Kiểu truyền	Lưuồng byte có kết nối	Datagram không kết nối
Độ tin cậy	Có thứ tự, kiểm soát lỗi và truyền lại	Không bảo đảm đến nơi, thứ tự hoặc không trùng lặp
Biên thông điệp	Không tự bảo toàn; ứng dụng phải quy định dấu kết thúc hoặc chiều dài	Mỗi DatagramPacket là một thông điệp
Lớp Java chính	ServerSocket và Socket	DatagramSocket và DatagramPacket
Tình huống phù hợp	Chat, truyền file, API cần chính xác	Khám phá dịch vụ, telemetry thời gian thực, multicast

3.3. Giao thức dòng dùng trong các ví dụ TCP

Mỗi yêu cầu và phản hồi là một dòng UTF-8, kết thúc bằng ký tự xuống dòng. Server không suy đoán ranh giới thông điệp từ số lần read.

Yêu cầu	Phản hồi	Ý nghĩa
PING	OK PONG	Kiểm tra server đang hoạt động
UPPER xin chào	OK XIN CHÀO	Đổi phần văn bản sang chữ hoa
TIME	OK 2026-09-13T10:30:00	Trả ngày giờ hiện tại của server
QUIT	OK BYE	Kết thúc phiên làm việc
Lệnh khác	ERR UNKNOWN_COMMAND	Thông báo yêu cầu không hợp lệ

3.4. Chuẩn bị môi trường

1. Cài JDK 17 trở lên và kiểm tra bằng lệnh `java --version` và `javac --version`.
2. Tạo thư mục dự án lab04 socket, đặt mỗi lớp public trong một file java cùng tên.
3. Chạy server trước, sau đó mở cửa sổ Terminal khác để chạy client.
4. Thực hành một máy bằng localhost. Khi thử hai máy, bảo đảm cùng mạng LAN và firewall cho phép cổng thực hành.
5. Không dùng địa chỉ IP cố định trong mã nguồn khi đề yêu cầu truyền host và port qua args.

```
lab04-socket/
src/
  network/HostInspector.java
  tcp/TcpCommandServer.java
```

```

tcp/TcpCommandClient.java
tcp/MultiClientTcpServer.java
udp/UdpEchoServer.java
udp/UdpEchoClient.java
evidence/
test-results.txt
screenshots/

```

4. CÁC HOẠT ĐỘNG THỰC HÀNH CÓ HƯỚNG DẪN

4.1. Khảo sát địa chỉ mạng

Mục tiêu: sử dụng `InetAddress` để phân giải hostname và nhận diện một số đặc điểm của địa chỉ trả về.

Mã nguồn `HostInspector`

```

package network;

import java.net.InetAddress;
import java.net.UnknownHostException;

public class HostInspector {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Usage: java network.HostInspector <hostname>");
            return;
        }

        try {
            InetAddress[] addresses = InetAddress.getAllByName(args[0]);
            System.out.println("Host: " + args[0]);

            for (InetAddress address : addresses) {
                System.out.println("- IP: " + address.getHostAddress());
                System.out.println("  Canonical: "
                    + address.getCanonicalHostName());
                System.out.println("  Loopback: "
                    + address.isLoopbackAddress());
                System.out.println("  Site local: "
                    + address.isSiteLocalAddress());
            }
        } catch (UnknownHostException e) {
            System.err.println("Không phân giải được host: " + args[0]);
        }
    }
}

```

Biên dịch và chạy

```

javac -d out src/network/HostInspector.java
java -cp out network.HostInspector localhost
java -cp out network.HostInspector example.com
java -cp out network.HostInspector host-khong-ton-tai.invalid

```

Yêu cầu thực hiện

- Ghi lại tất cả địa chỉ trả về cho localhost và một hostname hợp lệ.
- Giải thích sự khác nhau giữa loopback và site local.
- Bổ sung thông báo cho biết địa chỉ là IPv4 hay IPv6 dựa trên kiểu `Inet4Address` hoặc `Inet6Address`.

- Không yêu cầu xác định lớp A, B hoặc C; nếu giảng viên yêu cầu thì phải ghi rõ đây là kiến thức lịch sử.

Kiểm thử tối thiểu

Ca kiểm thử	Đầu vào	Kết quả mong đợi
Hợp lệ cục bộ	localhost	Có ít nhất một địa chỉ và Loopback bằng true
Hợp lệ bên ngoài	example.com	Hiển thị một hoặc nhiều IP nếu có kết nối DNS
Không tồn tại	host-khong-ton-tai.invalid	Thông báo lỗi rõ ràng, không in stack trace dài
Thiếu args	Không truyền tham số	Hiển thị cú pháp sử dụng và kết thúc

4.2. TCP client server theo giao thức dòng

Mục tiêu: tạo kết nối TCP, gửi nhiều lệnh trên cùng một socket và đóng phiên bằng lệnh QUIT.

TcpCommandServer

```
package tcp;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;
import java.util.Locale;

public class TcpCommandServer {
    private static final int PORT = 5000;

    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(PORT)) {
            System.out.println("TCP server listening on port " + PORT);

            while (true) {
                try (Socket socket = server.accept()) {
                    serve(socket);
                } catch (IOException e) {
                    System.err.println("Lỗi phiên client: " + e.getMessage());
                }
            }
        } catch (IOException e) {
            System.err.println("Không mở được server: " + e.getMessage());
        }
    }

    static void serve(Socket socket) throws IOException {
        try (BufferedReader in = new BufferedReader(new InputStreamReader(
            socket.getInputStream(), StandardCharsets.UTF_8));
            PrintWriter out = new PrintWriter(new OutputStreamWriter(
            socket.getOutputStream(), StandardCharsets.UTF_8), true)) {
            String request;
            while ((request = in.readLine()) != null) {
                String response = process(request);
                out.println(response);
                if (request.equalsIgnoreCase("QUIT")) break;
            }
        }
    }
}
```

```

    }
}

static String process(String request) {
    String trimmed = request.trim();
    if (trimmed.equalsIgnoreCase("PING")) return "OK PONG";
    if (trimmed.equalsIgnoreCase("TIME")) {
        return "OK " + LocalDateTime.now();
    }
    if (trimmed.equalsIgnoreCase("QUIT")) return "OK BYE";
    if (trimmed.regionMatches(true, 0, "UPPER ", 0, 6)) {
        return "OK " + trimmed.substring(6).toUpperCase(Locale.ROOT);
    }
    return "ERR UNKNOWN_COMMAND";
}
}

```

TcpCommandClient

```

package tcp;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class TcpCommandClient {
    public static void main(String[] args) {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;

        try (Socket socket = new Socket(host, port);
            BufferedReader console = new BufferedReader(
                new InputStreamReader(System.in, StandardCharsets.UTF_8));
            BufferedReader in = new BufferedReader(new InputStreamReader(
                socket.getInputStream(), StandardCharsets.UTF_8));
            PrintWriter out = new PrintWriter(new OutputStreamWriter(
                socket.getOutputStream(), StandardCharsets.UTF_8), true)) {

            System.out.println("Connected. Enter PING, TIME, UPPER text, QUIT");
            String request;
            while ((request = console.readLine()) != null) {
                out.println(request);
                String response = in.readLine();
                if (response == null) {
                    System.out.println("Server closed the connection");
                    break;
                }
                System.out.println("Server: " + response);
                if (request.equalsIgnoreCase("QUIT")) break;
            }
        } catch (NumberFormatException e) {
            System.err.println("Port phải là số nguyên");
        } catch (IOException e) {
            System.err.println("Lỗi kết nối: " + e.getMessage());
        }
    }
}

```

```
}
```

Lệnh biên dịch và chạy

```
javac -d out src/tcp/TcpCommandServer.java src/tcp/TcpCommandClient.java
java -cp out tcp.TcpCommandServer
java -cp out tcp.TcpCommandClient localhost 5000
```

Trình tự chạy và quan sát

1. Terminal 1: biên dịch hai lớp, sau đó chạy tcp.TcpCommandServer.
2. Terminal 2: chạy tcp.TcpCommandClient localhost 5000.
3. Lần lượt gửi PING, UPPER xin chào, TIME, ABC và QUIT.
4. Chạy hai client cùng lúc và ghi nhận hạn chế của server phục vụ tuần tự.

Kết quả mong đợi: mỗi yêu cầu hợp lệ nhận đúng một phản hồi; lệnh sai nhận ERR; QUIT đóng client nhưng server vẫn tiếp tục lắng nghe.

4.3. TCP server phục vụ đồng thời nhiều client

Mục tiêu: thay server tuần tự bằng thread pool, đồng thời tái sử dụng phương thức xử lý giao thức ở Hoạt động 2.

MultiClientTcpServer

```
package tcp;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MultiClientTcpServer {
    private static final int PORT = 5000;
    private static final int MAX_CLIENTS = 20;

    public static void main(String[] args) {
        ExecutorService pool = Executors.newFixedThreadPool(MAX_CLIENTS);

        try (ServerSocket server = new ServerSocket(PORT)) {
            System.out.println("Multi-client server on port " + PORT);

            while (true) {
                Socket socket = server.accept();
                pool.submit(() -> {
                    String client = String.valueOf(
                        socket.getRemoteSocketAddress());
                    System.out.println("Connected: " + client);
                    try (socket) {
                        TcpCommandServer.serve(socket);
                    } catch (IOException e) {
                        System.err.println("Client " + client
                            + " failed: " + e.getMessage());
                    } finally {
                        System.out.println("Disconnected: " + client);
                    }
                });
            }
        }
    }
}
```

```

        } catch (IOException e) {
            System.err.println("Server error: " + e.getMessage());
        } finally {
            pool.shutdown();
        }
    }
}

```

Kiểm thử đồng thời

1. Dừng TcpCommandServer rồi chạy MultiClientTcpServer.
2. Mở ba Terminal client và kết nối tới cùng cổng 5000.
3. Tại mỗi client gửi ít nhất ba lệnh, giữ các phiên cùng hoạt động.
4. Đóng đột ngột một client và xác nhận hai client còn lại không bị ảnh hưởng.
5. Giải thích vì sao fixed thread pool an toàn hơn việc tạo Thread không giới hạn.

Khi chạy trên hai máy

Bước	Máy server	Máy client
1	Xác định IPv4 riêng bằng ipconfig, ip addr hoặc ifconfig	Xác nhận hai máy cùng mạng LAN
2	Cho phép TCP port 5000 qua firewall	Không cần mở cổng lắng nghe
3	Chạy MultiClientTcpServer	Chạy TcpCommandClient serverIP 5000
4	Quan sát địa chỉ remote của client	Gửi PING và QUIT

Không tắt toàn bộ firewall. Chỉ tạo quy tắc cho đúng chương trình hoặc cổng thực hành và xóa quy tắc sau khi hoàn tất.

4.4. UDP request response

Mục tiêu: gửi một datagram, đọc đúng số byte thực nhận và xử lý trường hợp không có phản hồi.

UdpEchoServer

```

package udp;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.nio.charset.StandardCharsets;
import java.util.Locale;

public class UdpEchoServer {
    private static final int PORT = 5001;

    public static void main(String[] args) {
        byte[] buffer = new byte[4096];

        try (DatagramSocket socket = new DatagramSocket(PORT)) {
            System.out.println("UDP server listening on port " + PORT);

            while (true) {
                DatagramPacket request =
                    new DatagramPacket(buffer, buffer.length);
                socket.receive(request);
            }
        }
    }
}

```

```

        String message = new String(request.getData(),
                                    request.getOffset(), request.getLength(),
                                    StandardCharsets.UTF_8);
        String text = "ACK " + message.toUpperCase(Locale.ROOT);
        byte[] responseData = text.getBytes(StandardCharsets.UTF_8);

        DatagramPacket response = new DatagramPacket(
            responseData, responseData.length,
            request.getAddress(), request.getPort());
        socket.send(response);
    }
} catch (IOException e) {
    System.err.println("UDP server error: " + e.getMessage());
}
}
}

```

UdpEchoClient

```

package udp;

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.net.SocketTimeoutException;
import java.nio.charset.StandardCharsets;

public class UdpEchoClient {
    public static void main(String[] args) throws Exception {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 5001;
        String message = args.length > 2 ? args[2] : "xin chào UDP";

        byte[] data = message.getBytes(StandardCharsets.UTF_8);
        InetAddress server = InetAddress.getByName(host);

        try (DatagramSocket socket = new DatagramSocket()) {
            socket.setSoTimeout(3000);
            socket.send(new DatagramPacket(data, data.length, server, port));

            byte[] buffer = new byte[4096];
            DatagramPacket response =
                new DatagramPacket(buffer, buffer.length);
            try {
                socket.receive(response);
                String text = new String(response.getData(),
                                        response.getOffset(), response.getLength(),
                                        StandardCharsets.UTF_8);
                System.out.println("Server: " + text);
            } catch (SocketTimeoutException e) {
                System.err.println("Hết 3 giây nhưng chưa nhận được phản hồi");
            }
        }
    }
}

```

Lệnh biên dịch và chạy

```
javac -d out src/udp/UdpEchoServer.java src/udp/UdpEchoClient.java
```

```
java -cp out udp.UdpEchoServer
java -cp out udp.UdpEchoClient localhost 5001 "xin chào UDP"
```

Kiểm thử

Ca kiểm thử	Thao tác	Kết quả mong đợi
Tiếng Việt	Gửi xin chào UDP	Nhận ACK XIN CHÀO UDP đúng dấu
Server chưa chạy	Chạy client trước	Thông báo timeout sau khoảng 3 giây
Sai port	Client gửi tới 5999	Timeout, chương trình không treo vô hạn
Nhiều client	Chạy ba client gần đồng thời	Mỗi client nhận phản hồi cho datagram của mình
Thông điệp dài	Gửi dữ liệu gần 4096 byte	Đánh giá giới hạn buffer và đề xuất kiểm tra kích thước

5. BÀI TẬP ĐỀ XUẤT

Các bài tập được sắp xếp từ cơ bản đến nâng cao. Giảng viên có thể chọn bốn đến sáu bài tùy thời lượng; Bài 5 và Bài 8 phù hợp làm bài tổng hợp.

5.1 Danh mục bài tập

Bài	Tên bài tập	Trọng tâm	Mức độ
1	Host và URI Inspector	InetAddress, URI, args	Cơ bản
2	Đổi chữ số thành chữ	TCP request response, validation	Cơ bản
3	Dịch vụ ngày giờ	TCP hoặc UDP, menu lệnh	Cơ bản
4	Máy tính từ xa	Thiết kế protocol, xử lý lỗi	Trung bình
5	Chat nhiều client	Thread pool, broadcast, đồng bộ	Trung bình
6	So sánh TCP và UDP	Đo thời gian, timeout, mất gói	Trung bình
7	Nhật ký tin nhắn	Multi client, file I/O, định danh client	Nâng cao
8	Truyền file an toàn	Metadata, byte stream, SHA 256	Nâng cao
9	Multicast thông báo	MulticastSocket, group membership	Nâng cao
10	Khám phá dịch vụ	UDP discovery kết hợp TCP service	Nâng cao

5.2 Yêu cầu chi tiết từng bài

Bài 1 Host và URI Inspector

- Nhận hostname và URI qua args; không gộp hai giá trị thành một chuỗi.
- In toàn bộ IP, loại IPv4 hoặc IPv6, loopback và site local.
- Dùng java.net.URI để in scheme, host, port, path, query và fragment.
- Xử lý thiếu tham số, URI sai và hostname không phân giải được.

Minh chứng: ba ca hợp lệ và ba ca lỗi. Không yêu cầu tải mã nguồn trang web.

Bài 2 TCP đổi chữ số thành chữ

- Client gửi một ký tự từ 0 đến 9; server trả về cách đọc tiếng Việt tương ứng.
- Nếu dữ liệu không phải đúng một chữ số, server trả ERR_INVALID_DIGIT.
- Cho phép client gửi nhiều yêu cầu cho tới khi nhập QUIT.
- Bắt buộc dùng UTF-8 và giao thức mỗi thông điệp một dòng.

Ca biên bắt buộc: 0, 9, chuỗi rỗng, 10, ký tự a và dữ liệu có khoảng trắng.

Bài 3 Dịch vụ ngày giờ TCP và UDP

- Server hỗ trợ DATE, TIME, DATETIME và QUIT đối với TCP.
- Cài đặt cùng tập lệnh trên UDP; UDP không cần lệnh QUIT cho phiên kết nối.
- Dùng DateTimeFormatter với khuôn dạng dd MM yyyy và HH mm ss.
- Viết đoạn so sánh hành vi khi server dừng giữa lúc client đang hoạt động.

Bài 4 Máy tính từ xa

Thiết kế yêu cầu theo dạng CALC toán_tử toán_hạng_1 toán_hạng_2. Ví dụ CALC + 100 200. Server hỗ trợ cộng, trừ, nhân, chia và trả một trong hai dạng OK kết_quả hoặc ERR mã_lỗi.

Tình huống	Phản hồi gợi ý
CALC + 100 200	OK 300
CALC / 10 0	ERR DIVIDE_BY_ZERO
CALC % 10 3	ERR UNSUPPORTED_OPERATOR
CALC + a 2	ERR INVALID_NUMBER
Thiếu thành phần	ERR INVALID_FORMAT

Bài 5 Chat TCP nhiều client

- Mỗi client chọn nickname duy nhất sau khi kết nối.
- Tin nhắn của một client được broadcast cho các client còn lại.
- Server dùng thread pool và cấu trúc thread safe để quản lý danh sách người gửi.
- Hỗ trợ các lệnh USERS, MSG nội_dung và QUIT.
- Khi một client ngắt bất thường, server loại client đó và tiếp tục hoạt động.

Bài 6 Thực nghiệm so sánh TCP và UDP

- Gửi 1000 thông điệp kích thước cố định bằng TCP và UDP trên localhost.
- Đo tổng thời gian và số phản hồi nhận được; lặp lại ít nhất năm lần.
- Không kết luận UDP luôn nhanh hơn chỉ từ một lần chạy.
- Ghi rõ môi trường, kích thước thông điệp, timeout và phương pháp đo.

Bài 7 Server lưu nhật ký tin nhắn

- Client gửi HELLO clientId trước khi gửi nội dung.
- Server lưu tin nhắn theo từng client vào data logs clientId.txt.
- Mỗi dòng gồm timestamp, địa chỉ remote và nội dung.

- Kết thúc bằng QUIT thay cho chuỗi HET nằm lẫn trong nội dung.
- Chỉ chấp nhận clientId gồm chữ, số, dấu gạch ngang và gạch dưới.

Bài 8 Truyền file TCP an toàn

- Client gửi tên file, kích thước kiểu long và mã SHA 256 trước phần dữ liệu.
- Server lưu file vào thư mục upload cố định và không chấp nhận đường dẫn do client chỉ định.
- Server đọc đúng số byte đã khai báo, không dùng EOF làm ranh giới duy nhất.
- Sau khi lưu, server tính lại SHA 256 và trả OK hoặc ERR HASH_MISMATCH.
- Kiểm thử file rỗng, file văn bản tiếng Việt, file ảnh và tên chứa chuỗi ../.

Bài 9 Multicast thông báo

- Sender gửi thông báo tới địa chỉ multicast 239.255.0.1 thuộc phạm vi quản trị cục bộ.
- Ít nhất hai receiver cùng tham gia group và nhận thông điệp.
- Chỉ rõ NetworkInterface khi joinGroup và rời group trước khi đóng socket.
- Ghi nhận trường hợp multicast bị chặn bởi Wi Fi, VPN hoặc firewall; luôn có phương án kiểm thử localhost hoặc mạng phòng lab.

Bài 10 UDP discovery kết hợp TCP service

- Client broadcast hoặc multicast DISCOVER_SERVICE qua UDP.
- Server trả SERVICE tên_dịch_vụ tcp_port phiên_bản.
- Client dùng địa chỉ nguồn của datagram phản hồi để kết nối tới TCP server.
- Thiết kế timeout, loại bỏ phản hồi trùng và hiển thị nhiều server nếu có.